جلسهٔ ۱ از ۵

چرخهٔ جدید تولید نرم‌افزار با کدنویسی شهودی

از پرامپت‌های (prompt) نامنظم تا مهندسی عامل‌محور (Agentic engineering)

بر پایهٔ مقالهٔ آدی عثمانی، شوبهام سابو و سوکراتیس کارتاکیس

نقشهٔ دوره

پنج جلسه، یک مسیر

۱ — چرخهٔ جدید تولید نرم‌افزار

چارچوب ذهنی: طیف، زمینه، مهار، نقش تازهٔ توسعه‌دهنده.

۲ — ابزارهای عامل و تعامل‌پذیری

عامل (agent) چگونه با جهان بیرون حرف می‌زند: ابزار، MCP، A2A.

۳ — مهارت‌های عامل

نشست، مهارت (skill) و حافظه؛ اقتصاد توکن (token) در مقیاس تولید.

۴ — امنیت و ارزیابی

سطح حملهٔ تازه، گاردریل‌ها، و سنجش کیفیت عامل.

۵ — توسعهٔ مبتنی بر مشخصات در سطح محصول

از نمونهٔ اولیه تا نرم‌افزاری که می‌شود به آن تکیه کرد.

وضعیت امروز

این دیگر آینده‌پژوهی نیست

٪۸۵

از توسعه‌دهندگان حرفه‌ای مرتب از عامل‌های کدنویسی استفاده می‌کنند

٪۵۱

هر روز از آن‌ها استفاده می‌کنند

٪۴۱

از کدهای تازه‌نوشته‌شده، تولید هوش مصنوعی است

مسیر این تحول: تکمیل خودکار ← پیشنهاد درون‌خطی ← گفت‌وگو ← عامل خودگردانی که ریپو (repository) را می‌خواند، تغییر می‌دهد، تست (test) می‌گیرد و پول‌ریکوئست (pull request) می‌سازد.

> بزرگ‌ترین تحول در مهندسی نرم‌افزار، یک زبان یا چارچوب تازه نیست؛ گذار از **نوشتن کد** به **بیان قصد** (intent) است — و سپردن این کار به سامانه‌های هوشمند که آن قصد را به نرم‌افزاری کارآمد تبدیل کنند.

تز اصلی جلسهٔ امروز

بخش یک

عامل چیست؟

یک مرور سریع، چون هر چیز دیگری در این دوره روی همین تعریف بنا می‌شود.

حلقهٔ عامل

ربات گفت‌وگو پاسخ می‌دهد و منتظر می‌ماند. عامل حلقهٔ خودش را می‌چرخاند: شما هدف را می‌دهید، او در هر گام تصمیم می‌گیرد قدم بعدی چیست.



نکته: این حلقه، قلب تپندهٔ هر عامل است. هر چیزی که در چهار جلسهٔ بعد می‌بینید، صورتی از همین حلقه (loop) است.

پنج جزء سازندهٔ هر عامل

۱

مدل

موتور استدلال: زمینه (context) را می‌خواند و تصمیم می‌گیرد گام بعد چه باشد.

۲

ابزارها

اتصال مدل به جهان: API، اجرای کد، پرس‌وجوی پایگاه داده، واگذاری (handoff) به عامل دیگر.

۳

حافظه

وضعیت: یادآوری تعامل‌های پیشین و قواعد پروژه، تا هرگز از صفر شروع نکند.

۴

ارکستراسیون

کدی که حلقه را می‌چرخاند: ساختن زمینه، فرستادن فراخوان‌ها، تصمیم به ادامه یا توقف.

۵

دیپلوی

آنچه نمونهٔ اولیه را به سرویس تبدیل می‌کند: میزبانی، هویت، رصدپذیری (observability) و زیرساخت تولید.

چرا این تفکیک مهم است؟

	ربات گفت‌وگو	عامل
واحد کار	یک پرسش، یک پاسخ	یک هدف، چندین گام
تصمیم‌گیرندهٔ گام بعد	انسان	خود سامانه
نسبت با جهان	فقط متن	کنش واقعی از راه ابزار
شرط پایان	پایان پاسخ	رسیدن به هدف یا برخورد به شرط توقف
ریسک	پاسخ نادرست	کنش نادرست روی سامانهٔ واقعی

پیامد: وقتی سامانه می‌تواند کنش کند، «کیفیت پاسخ» کافی نیست؛ به مهار، گاردریل (guardrail) و ارزیابی (evaluation) نیاز داریم.

بخش دو

طیف: از کدنویسی شهودی تا مهندسی عامل‌محور

پرسش درست این نیست که «آیا از هوش مصنوعی استفاده می‌کنید؟» بلکه این است که «چقدر ساختار، راستی‌آزمایی و قضاوت انسانی پیرامون خروجی آن هست؟»

کدنویسی شهودی چیست؟

در فوریهٔ ۲۰۲۵، آندری کارپثی شیوه‌ای را توصیف کرد که خیلی‌ها بی‌آنکه نامی برایش داشته باشند، به آن عمل می‌کردند:

> کاملاً تسلیم حس‌وحال شو، رشد شتابان توانایی‌های هوش مصنوعی را بپذیر و فراموش کن که اصلاً کدی وجود دارد.

- خواسته را به زبان طبیعی توصیف می‌کنی.
- خروجی مدل را می‌پذیری، بدون آنکه لزوماً بخوانی‌اش.
- وقتی چیزی خراب شد، پیام خطا را کپی می‌کنی و می‌گویی «درستش کن».

واژه‌ای که از فرط گستردگی، معنایش را باخت

مسئله

ظرف چند ماه، «کدنویسی شهودی (Vibe coding)» به هر گردش‌کار (workflow) مبتنی بر هوش مصنوعی گفته شد. آیا مهندس ارشدی که یک ویژگی دقیقاً مشخص‌شده را با کمک هوش مصنوعی پیاده می‌کند، «شهودی» کار می‌کند؟

پاسخ

تا اوایل ۲۰۲۶ خود کارپثی پذیرفت که تعریف اولیه‌اش محدود بوده است و اصطلاح **مهندسی عامل‌محور** را برای سرِ ساختارمند این طیف پیشنهاد کرد.

چرا واژه مهم است: اگر به مدیر فنی بگویید تیم شما سامانهٔ پرداخت را «شهودی» می‌نویسد، باید زنگ خطر به صدا دربیاید — و درست هم هست. اگر بگویید «مهندسی عامل‌محور» می‌کنید، گفت‌وگو از بنیاد فرق می‌کند.

طیف در شش بُعد

بُعد	کدنویسی شهودی	کمک‌گرفتن ساختارمند	مهندسی عامل‌محور
بیان قصد	پرامپت‌های گفتاری و سرِدستی	پرامپت‌های دقیق با مثال و قید (constraint)	مشخصات (spec) رسمی، سند معماری، فایل‌های حافظه (memory)
راستی‌آزمایی	«به‌نظر کار می‌کند؟»	تست دستی و وارسی نمونه‌ای	مجموعه‌تست خودکار، دروازه‌های (gate) `CI/CD`، داوران مدل‌زبانی
خواندن کد	ممکن است اصلاً خوانده نشود	بازبینی گزینشی مسیرهای حساس	بازبینی کامل معماری؛ جزئیات پیاده‌سازی با عامل
خطا	پیام خطا را برگردان به مدل	انسان ریشه را می‌یابد، عامل اصلاح می‌کند	عامل درون مرزهای تعریف‌شده خودتشخیص می‌دهد
دامنهٔ مناسب	نمونهٔ اولیه، اسکریپت، هکاتون	افزودن ویژگی به کدبیس موجود	سامانهٔ تولیدی، توسعه در مقیاس تیم
ریسک	بالا؛ برای کد یک‌بارمصرف پذیرفتنی	متوسط؛ قضاوت انسانی در ایستگاه‌های کلیدی	پایین؛ راستی‌آزمایی نظام‌مند در هر مرحله

تمایزدهندهٔ اصلی: چگونه راستی‌آزمایی می‌کنید

تست‌ها

بخش **قطعی (deterministic)** سامانه را می‌سنجند: این ورودی باید آن خروجی را بدهد. داورشان کد است.

ارزیابی‌ها

بخش **غیرقطعی (not deterministic)** را می‌سنجند: آیا عامل مسیر (trajectory) درستی رفت؟ ابزار درست را برگزید؟ پاسخ نهایی به حد کیفی رسید؟ داورشان دادگان برچسب‌خورده، سنجه‌ها (rubric) و داوران مدل‌زبانی‌اند.

قاعدهٔ سخت: بدون هر دو، کاری که می‌کنید کدنویسی شهودی است — هرقدر هم پرامپت‌هایتان پیچیده و حرفه‌ای باشد.

بخش سه

مهندسی زمینه: مهارت واقعی

کیفیت کد تولیدشده، کمتر به زیرکی پرامپت شما بستگی دارد و بیشتر به کیفیت زمینه‌ای که فراهم کرده‌اید وابسته است.

شش نوع زمینه

دستورها (Instructions)

نقش عامل، هدف‌ها و مرزهای عملیاتی‌اش.

دانش (Knowledge)

سندهای بازیابی‌شده، نمودار معماری، دادهٔ حوزه.

حافظه (Memory)

کوتاه‌مدت: چه گذشت. بلندمدت: این پروژه چیست.

مثال‌ها (Examples)

نمونه‌های رفتاری و الگوهای مرجع درون کدبیس.

ابزارها (Tools)

تعریف دقیق APIها، اسکریپت‌ها و سرویس‌های قابل فراخوانی.

گاردریل‌ها (Guardrails)

قیدهای سخت، قواعد قالب‌بندی، اعتبارسنجی‌های ایمنی.

پرسش راهنما: «یک هم‌تیمی تازه‌وارد چه چیزهایی باید بداند تا مفید باشد؟ و چطور آن دانش را به شکلی بنویسم که عامل هم بتواند استفاده کند؟»

زمینهٔ ایستا در برابر زمینهٔ پویا

ایستا — همیشه بارگذاری می‌شود

سیستم پرامپت، فایل‌های قاعده (`AGENTS.md`، `CLAUDE.md`، `GEMINI.md`)، حافظهٔ سراسری، تعریف شخصیت.

هزینه: هر توکن آن در **هر** تعامل حاضر است، چه به کار بیاید چه نه.

پویا — هنگام نیاز بارگذاری می‌شود

دستور مهارت‌هایی که با کار جاری هم‌خوان‌اند، نتیجهٔ ابزارها، سندهای بازیابی‌شده، بخش اخیر تاریخچهٔ نشست (session).

هزینه: فقط وقتی می‌پردازید که واقعاً لازم باشد.

این یک تصمیم معماری است: زمینهٔ ایستای زیاد، توکن هدر می‌دهد و سیگنال را رقیق می‌کند؛ زمینهٔ ایستای کم، یعنی عامل قواعد حیاتی را فراموش می‌کند. این مرز باید مثل هر پیکربندی دیگری بازبینی و نسخه‌بندی شود.

مهارت‌های عامل: قوی‌ترین الگوی زمینهٔ پویا

بسته‌های ساختارمند و قابل‌حمل از دانش رویه‌ای (procedural knowledge)، که عامل فقط وقتی آن‌ها را بارگذاری می‌کند که کار اقتضا کند.

آغاز
عامل فقط فراداده‌های سبک را می‌بیند

←

تطبیق
کار با یک مهارت هم‌خوان می‌شود

←

بارگذاری
دستور کامل مهارت خوانده می‌شود

←

عمق
مرجع تفصیلی فقط در صورت نیاز

نتیجه: عامل می‌تواند ده‌ها توانایی تخصصی حمل کند و فقط برای همانی که همین حالا به کار گرفته، توکن بپردازد — یک عامل عمومی سبک که به‌اقتضا متخصص می‌شود.

مهارت‌ها چهار درد قدیمی را درمان می‌کنند

پوسیدگی زمینه

سیستم پرامپت‌هایی که بیش از حد در آن‌ها مطلب گنجانده شده، سیگنال را از بین می‌برند.

نبود حافظهٔ رویه‌ای

مدل‌های زبانی «چگونه انجامش بدهیم» را به خاطر نمی‌سپارند.

سربار معماری چندعاملی

برای هر تخصص، یک عامل جدا ساختن گران و شکننده است.

نبود قابلیت حمل

دانش تیم نباید به یک ابزار یا ارائه‌دهندهٔ خاص قفل شود.

پل به جلسهٔ ۳: طراحی، نوشتن و ارزیابی مهارت‌ها، مدیریت نشست و حافظه، و اقتصاد توکن را در جلسهٔ سوم به‌تفصیل می‌بینیم.

بخش چهار

چرخهٔ جدید تولید نرم‌افزار

وقتی گلوگاه از «دست انسان» به «ذهن انسان» منتقل می‌شود، خودِ فرایند باید بازطراحی شود.

فشرده‌سازی، اما نامتوازن

آنچه به‌شدت کوتاه می‌شود

پیاده‌سازی: کاری که هفته‌ها می‌برد، اکنون ساعت‌ها.

آنچه با سرعت انسان می‌ماند

نیازمندی، معماری و راستی‌آزمایی.

نتیجه، نسخهٔ سریع‌ترِ همان چرخهٔ قدیمی نیست؛ گردش‌کاری متفاوت است که در آن مرز فازها محو می‌شود، تکرارها از هفته به دقیقه می‌رسد، و توسعه‌دهنده از «پیاده‌ساز اصلی» به «طراح سامانه و داور (judge) کیفیت» بدل می‌شود.

فاز به فاز — نیازمندی و معماری

نیازمندی و برنامه‌ریزی

- تولید داستان کاربر از خلاصهٔ محصول، کشف حالت‌های مرزی، ساخت اسکیمای `API` از توضیح زبان طبیعی.

  حلقهٔ «نیازمندی تا نمونهٔ اولیه» تقریباً به صفر می‌رسد.

- نیازمندی دیگر سندی نیست که بین تیم‌ها دست‌به‌دست شود؛ گفت‌وگویی است که هم‌زمان مشخصات و پیاده‌سازی اولیه تولید می‌کند.

طراحی و معماری

- سرسخت‌ترین فاز در برابر خودکارسازی؛ چون معماری یعنی موازنه: سازگاری در برابر دسترس‌پذیری، پیچیدگی در برابر انعطاف، ساخت در برابر خرید.
- هوش مصنوعی در **اجرای** تصمیم معماری عالی است، نه در **گرفتن** آن.

فاز به فاز — پیاده‌سازی و تست

سود بهره‌وری

پیمایش‌های صنعتی ۲۵ تا ۳۹ درصد بهبود گزارش می‌کنند.

تصویر کامل‌تر

پژوهش METR: توسعه‌دهندگان باتجربه در برخی کارها ۱۹٪ **کندتر** شدند — به‌دلیل وقتِ راستی‌آزمایی و اصلاح خروجی.

هوش مصنوعی کار پیاده‌سازی را حذف نمی‌کند؛ آن را از «نوشتن» به «بازبینی، هدایت و راستی‌آزمایی» تبدیل می‌کند.

در تست: هم خروجی را بسنجید (آیا کامپایل می‌شود؟ تست‌ها سبزند؟) هم مسیر را (آیا گام‌های راستی‌آزمایی طی شد؟). خروجی روانی که مسیر را میان‌بر زده، از خروجی با خطای آشکار خطرناک‌تر است.

فاز به فاز — بازبینی، دیپلوی، نگه‌داری

- **بازبینی کد:** هوش مصنوعی به‌عنوان بازبین نوبت اول — یافتن اشکال، نقض سبک، آسیب‌پذیری و مسائل کارایی پیش از آنکه انسان کد را ببیند. جای انسان را نمی‌گیرد، بار شناختی او را کم می‌کند.
- **دیپلوی (deploy):** عامل‌ها سلامت انتشار را رصد می‌کنند، انتشار مشکل‌دار را برمی‌گردانند و ریسک را بر اساس دامنهٔ تغییر پیش‌بینی می‌کنند.
- **نگه‌داری — بزرگ‌ترین تغییری که کمتر دیده شده:** کدبیس‌هایی که کسی جرئت دست‌زدن به آن‌ها را نداشت، حالا قابل فهم و بازسازی‌اند. مهاجرت چارچوب، به‌روزرسانی APIهای منسوخ و نوسازی مجموعه‌تست‌ها — کارهایی که پیش‌تر هرگز انجام نمی‌شدند.

جلسهٔ ۵: وقتی حجم پول‌ریکوئست‌ها همپای خروجی عامل‌ها زیاد می‌شود، بازبینی انسانی چطور باید تغییر کند.

مدل کارخانه

محصول اصلی کار توسعه‌دهنده دیگر خودِ کد نیست، بلکه سامانه‌ای است که کد را تولید می‌کند.

مشخصات و زمینه

تعریف آنچه باید ساخته شود.

عامل‌ها

ترجمهٔ مشخصات به پیاده‌سازی.

تست‌ها و دروازه‌های کیفیت

راستی‌آزمایی درستی.

حلقه‌های بازخورد

بازگرداندن خطاها به عامل برای اصلاح.

گاردریل‌ها

ایمن و قابل‌پیش‌بینی نگه‌داشتن رفتار عامل.

قاعده: مدیر کارخانه قطعه‌ها را خودش سرِهم نمی‌کند؛ خط مونتاژ را طراحی می‌کند و از کیفیت محصول اطمینان می‌یابد. به‌جای دستورالعمل گام‌به‌گام، «معیار موفقیت» را به عامل‌ها بدهید و بگذارید تکرار کنند.

بخش پنج

مهندسی مهار

اگر توسعه‌دهنده مدیر کارخانه است، مدل فقط موتوری خام در کارگاه است. موتور به‌تنهایی خودرو نمی‌سازد.

عامل = مدل + مهار

وسوسه می‌شویم مدل را همان کل سامانه بدانیم: مدل تازه بیاید، عامل باهوش‌تر می‌شود؛ مدل قدیمی باشد، عامل ضعیف‌تر. این برداشت غلط است و به سرمایه‌گذاری‌های اشتباه می‌انجامد.

> مدل خام به‌تنهایی عامل نیست؛ وقتی به عامل تبدیل می‌شود که **مهار (harness)**، وضعیت، امکان اجرای ابزار، حلقهٔ بازخورد و قیدهای الزام‌آور را در اختیارش بگذارد.

رفتاری که هنگام کار با `Claude Code`، `Cursor`، `Codex` یا `Cline` تجربه می‌کنید، بیش از آنکه محصول مدل زیرین باشد، محصول مهار است.

داخل مهار چیست؟

دستورها و فایل‌های قاعده

`AGENTS.md`، فایل‌های مهارت، پرامپت‌های زیرعامل‌ها (subagent): عامل کیست و چه کارهایی برایش ممنوع است.

ابزارها

توابع، سرورهای MCP و APIها، به‌همراه توضیحاتی که می‌گوید کِی و چگونه از آن‌ها استفاده شود.

سندباکس و محیط اجرا

کد عامل کجا اجرا می‌شود، به چه چیزهایی دسترسی دارد و به چه چیزهایی ندارد.

منطق ارکستراسیون

راه‌اندازی زیرعامل‌ها، مسیریابی (routing) میان مدل‌ها، واگذاری کار میان عامل‌های متخصص و قواعد فعال‌شدن هرکدام.

گاردریل‌ها و قلاب‌ها

کد قطعی که در نقاط مشخص چرخهٔ عمر اجرا می‌شود: پیش از فراخوان ابزار، پس از ویرایش فایل، پیش از کامیت (commit).

رصدپذیری

گزارش، ردیابی، ارزیابی، سنجش هزینه و تأخیر (latency). بدون آن نمی‌دانیم عامل خوب کار می‌کند یا بی‌سروصدا منحرف شده.

توجه: این سطح گسترده، سطحِ **تیم شماست**، نه ارائه‌دهندهٔ مدل.

مهار در چهار فاز چرخهٔ عمر

فاز	اجزای فعال مهار	کاری که انجام می‌شود
نیازمندی، برنامه، معماری پیکربندی مهار	دستورها و فایل‌های قاعده	تعریف قیدهای معماری، مشخص‌کردن ابزارهای در دسترس، و قواعدی که عامل حق شکستن‌شان را ندارد.
پیاده‌سازی اجرای مهار	سندباکس، محیط اجرا، ابزارها	کد درون سندباکس (sandbox) ایزوله اجرا می‌شود؛ خواندن فایل یا جست‌وجو فقط از راه ابزارهای مهار.
تست و تضمین کیفیت حلقهٔ بازخورد	منطق ارکستراسیون، گاردریل‌ها	تست در محیط اجرا می‌رود؛ اگر شکست بخورد، ارکستراسیون (orchestration) خطا را می‌گیرد و به مدل برمی‌گرداند: «دوباره تلاش کن».
بازبینی، دیپلوی، نگه‌داری رصد مهار	قلاب‌ها، رصدپذیری	قلاب (hook) قطعی جلوی کامیتِ گذرواژهٔ سخت‌کدشده را می‌گیرد؛ لایهٔ رصد هزینه، تأخیر و انحراف (drift) را دنبال می‌کند.

اثر مهار، عددی است

Top 30 → Top 5

در بنچمارک (benchmark) Terminal Bench 2.0، یک تیم فقط با تغییر مهار — بدون هیچ تغییری در مدل — از بیرون سی عامل برتر به جمع پنج عامل برتر رسید.

+13.7

پژوهشی در LangChain با دست‌کاری فقط سیستم پرامپت، ابزارها و میان‌افزار — روی مدلی ثابت — امتیاز را ۱۳٫۷ واحد بالا برد.

تفاوت «کدنویسی شهودی» و «مهندسی عامل‌محور» در ابزار نیست؛ یک توسعه‌دهنده می‌تواند با عامل یکسان، هر دو کار را بکند. تفاوت در این است که مهار را چقدر آگاهانه پیکربندی کرده‌اید.

> وقتی عامل کاری را اشتباه انجام می‌دهد، اولین غریزه سرزنش مدل است. اما اغلب، رد خطا به یک ابزار غایب، یک قاعدهٔ مبهم، یک گاردریل نبوده یا کانتکست ویندوای پر از نویز می‌رسد.

بیشتر خطاهای عامل، اگر صادقانه بررسی شوند، در واقع خطای پیکربندی‌اند.

بخش شش

نقش در حال تحول توسعه‌دهنده

دو حالت کاری که توسعه‌دهندگان روان بین‌شان جابه‌جا می‌شوند.

رهبر ارکستر: هدایت مستقیم و هم‌زمان

چه شکلی است

در ویرایشگر نوشته‌شدن کد را می‌بینی و با پرامپت و اصلاح هدایتش می‌کنی. کنترل دقیقی بر هر چیزی که نوشته می‌شود.

کِی مناسب است

منطق پیچیده، دیباگ (debug) دشوار، یا کدبیسی ناآشنا که باید هر تغییر را همان لحظه بفهمی.

ابزارهای نمونه

GitHub Copilot، Gemini Code Assist، Cursor، Windsurf.

ریسک

اگر تک‌تک ضربه‌های کلید را خودت هدایت کنی، خودت گلوگاه می‌شوی و سرعت کار چندان بیشتر نمی‌شود.

هماهنگ‌کننده: واگذاری ناهم‌زمان به چند عامل

چه شکلی است

هدف را تعریف می‌کنی، به عامل‌ها می‌سپاری و نتیجه را بازبینی می‌کنی. عامل‌ها در پس‌زمینه و به‌طور موازی روی بخش‌های مختلف کدبیس کار می‌کنند.

کِی مناسب است

کارهایی که به‌خوبی تعریف شده‌اند: رفع اشکال شناخته‌شده، پیاده‌سازی ویژگی بر پایهٔ الگوی موجود، مهاجرت کدبیس، تولید مجموعه‌تست.

مهارت لازم عوض می‌شود: به‌جای تسلط عمیق بر نحو و ریزه‌کاری‌های زبان، چهار توانایی دیگر تعیین‌کننده می‌شوند.

مشخصات‌نویسی

تعریف کار آن‌قدر دقیق که ابهامی نماند.

تجزیه

تقسیم کار بزرگ به بخش‌هایی با اندازهٔ مناسب برای عامل.

ارزیابی

سنجش سریع اینکه خروجی به حد کیفی رسیده یا نه.

طراحی سامانه

ساختن قیدها، تست‌ها و حلقه‌های بازخوردی که کار عامل را بهره‌ور نگه می‌دارند.

عامل‌های کدنویسی در روزِ کاری، سه جا ظاهر می‌شوند

در ویرایشگر

تکمیل درون‌خطی، پنل گفت‌وگو، آگاهی از کل کدبیس. جایی که تمرکز کار حفظ می‌شود.

`Copilot · Cursor · Windsurf`

در ترمینال

هدف را به زبان ساده می‌دهی؛ عامل در کل کدبیس کار می‌کند، ابزار و تست اجرا می‌کند و بر اساس نتیجه تکرار می‌کند.

`Claude Code · Codex CLI · Antigravity · Cline`

در پس‌زمینه

کار را می‌گیرد، ساعت‌ها در سندباکس ابری خودگردان (autonomous) کار می‌کند و معمولاً یک پول‌ریکوئست تحویل می‌دهد.

`Jules · Copilot agent mode · background agents`

انتخاب درست: به ماهیت کار بستگی دارد، نه به اینکه کدام نوع عامل خودگردان‌تر است. بیشتر توسعه‌دهندگان هر سه را در یک روز به کار می‌گیرند.

مسئلهٔ ۸۰ درصد

عامل‌ها حدود ۸۰٪ کد یک ویژگی را به‌سرعت تولید می‌کنند. ۲۰٪ باقی‌مانده — یعنی حالت‌های مرزی، مدیریت خطا، نقاط اتصال به بخش‌های دیگر سامانه، و جزئیات حساسی که درستی کار به آن‌ها بستگی دارد — شناخت عمیقی از بستر پروژه می‌خواهد که مدل‌های امروز اغلب ندارند.

ماهیت خطاها عوض شده: از اشتباه نحوی به خطای مفهومیِ پنهان — پیش‌فرض غلط دربارهٔ منطق کسب‌وکار، نپرسیدن سؤال دربارهٔ نیازمندی مبهم، نادیده‌گرفتن حالت مرزی، و تصمیم معماری‌ای که در درازمدت بار نگه‌داری پنهانی می‌سازد. تشخیص این خطاها سخت‌تر است، دقیقاً چون کد «درست به‌نظر می‌رسد» و حتی ممکن است تست‌های ساده را هم پاس کند.

رویکرد درست: هوش مصنوعی را برای کارهایی به کار بگیرید که در آن‌ها خوب است (پیاده‌سازی سریع کارهایی که به‌خوبی تعریف شده‌اند) و توجه خودتان را برای جاهایی نگه دارید که هوش مصنوعی در آن‌ها ضعیف است (نیازمندی مبهم، موازنه‌های معماری، راستی‌آزمایی درستی کار).

وقتی خودِ محصول یک عامل است

تا اینجا از عامل‌ها برای ساختن نرم‌افزار استفاده کردیم. اما اگر چیزی که می‌خواهید بسازید خودش یک عامل باشد، چه؟ مثلاً ربات پشتیبانی که درخواست بازپرداخت را رسیدگی می‌کند، دستیار پژوهشی که منابع را با هم مقایسه می‌کند، یا ابزاری داخلی که رعایت مقررات را پایش می‌کند.

```
# one-time setup
uvx google-agents-cli setup

# then, inside your own coding agent:
> Build a support agent that answers questions from our docs.
> Evaluate it on the FAQ dataset.
> Deploy it to Agent Engine.
```

پشت همین سه جمله: ساخت اسکلت پروژه، نوشتن کد، تولید مجموعهٔ ارزیابی، اجرای آن، دیپلوی و گزارش نتیجه. نمونهٔ اولیهٔ دیروزِ روی لپ‌تاپ، امروز بدون بازنویسی به عامل تولیدی بدل می‌شود.

بخش هفت

هزینه و صرفهٔ اقتصادی توسعه با هوش مصنوعی

بحث معمولاً فقط حول «سرعت توسعه‌دهنده» می‌چرخد. اما معیار مهم‌تر برای مدیران فنی، هزینهٔ کل مالکیت است.

بدهی پنهان کدنویسی شهودی

سرمایه‌گذاری اولیه تقریباً صفر است؛ اما بار عملیاتی انباشته می‌شود.

هدررفت توکن

ریختن فایل‌های حجیم و بی‌ساختار در کانتکست ویندو (context window) و پرامپت‌های پیاپی برای اصلاح اشتباه‌های بررسی‌نشده: چرخه‌ای پرهزینه که کار در آن به‌ندرت در دفعهٔ اول درست درمی‌آید.

هزینهٔ پنهان نگه‌داری

کدی که ساختار منسجمی ندارد؛ شش ماه بعد، مهندس باید روزها وقت بگذارد تا از آن سر دربیاورد.

هزینهٔ رفع مشکلات امنیتی

بدون مهاری برای ارزیابی خودکار، تولید سریع کد یعنی تولید سریع آسیب‌پذیری. رفع یک نقص امنیتی در محیط تولید، چندین برابر گران‌تر از رفع آن در مرحلهٔ طراحی است.

ترجمهٔ مالی: سرمایه‌گذاری اولیهٔ کم، هزینهٔ عملیاتی زیاد.

سرمایه‌گذاری مهندسی عامل‌محور

هزینهٔ اولیهٔ بیشتر

طراحی اسکیمای API، ساخت مجموعه‌تست قطعی، و مهم‌تر از همه سازمان‌دادن به زمینهٔ عامل — پیش از تولید نخستین خط کد تولیدی.

هزینهٔ کمتر برای هر ویژگی تازه

هزینهٔ افزودن و نگه‌داشتن هر ویژگیِ تازه به‌شدت کاهش می‌یابد. عامل درون «کارخانه‌ای» حکمرانی‌شده کار می‌کند: خروجی از پیش تست‌شده و هم‌راستا با استانداردهاست.

مهندسی زمینه (Context engineering) به‌عنوان اهرم مالی: مدل‌ها بابت هر تکه اطلاعاتی که می‌فرستید پول می‌گیرند. فرستادن ریپوی ۱۰۰,۰۰۰ توکنی در هر پرامپت، وقتی تعداد پرامپت‌ها زیاد شود، اقتصادی نیست. زمینهٔ فشرده و پرسیگنال، احتمال درست‌شدن کار در دفعهٔ اول را بالا می‌برد و چرخه‌های پرهزینهٔ آزمون‌وخطا را از بین می‌برد.

مسیریابی هوشمند مدل

نوع کار	مدل مناسب	چرا
نیازمندی، معماری، پیاده‌سازی اولیه	مدل بزرگ و پیشرو	استدلال پیچیده ارزش هزینه را دارد
تولید تست، بازبینی کد، پایش CI/CD	مدل کوچک‌تر، سریع‌تر و بسیار ارزان‌تر	کار قطعی‌تر و کم‌پیچیدگی‌تر است

ضدالگو: در گردش‌کار شهودی، یک مدل عظیم برای همه‌چیز به کار می‌رود — حتی برای اصلاح یک غلط تایپی یا نوشتن یک تست واحد ساده. این یعنی پرداخت گران‌ترین نرخ برای ساده‌ترین کار.

کارگاه

ده خط که کیفیت خروجی عامل را عوض می‌کند

همین حالا برای ریپو خودتان یک `AGENTS.md` بنویسید — چهار بخش، هر کدام دو تا سه خط.

۱

پشتهٔ فناوری

زبان، چارچوب، نسخه‌ها، ابزار ساخت.

۲

قراردادها

سبک کد، ساختار پوشه‌ها، الگوی نام‌گذاری.

۳

قواعد سخت

کارهایی که عامل هرگز نباید بکند.

۴

گردش کار

فرمان تست، فرمان قالب‌بندی، انتظار از پیام کامیت.

عادت کلیدی: هر بار عامل کاری کرد که نباید، یک قاعدهٔ تازه اضافه کنید. فایل قاعده، حافظهٔ نهادی تیم شماست.

سه اصل ماندگار

۱

در مقیاس بزرگ، ساختار جواب می‌دهد، نه حس‌وحال

کدنویسی شهودی برای کاوش و نمونه‌سازی رویکرد درستی است. اما برای نرم‌افزاری که سازمان به آن تکیه می‌کند، انضباط عامل‌محور یک ضرورت است.

۲

هوش مصنوعی فرهنگ مهندسی را تقویت می‌کند

مثل یک اهرم عمل می‌کند و هم نقاط قوت و هم نقاط ضعف شما را چند برابر می‌کند.

۳

نقش انسان کوچک نمی‌شود، بلکه تغییر می‌کند

مهارت‌های تعیین‌کننده از پیاده‌سازی به قضاوت جابه‌جا می‌شوند: از نوشتن کد به طراحی سامانه‌هایی که کد تولید می‌کنند.

تولید حل شده است. راستی‌آزمایی، قضاوت و جهت‌دهی، پیشهٔ تازه‌اند.

تکلیف تا جلسهٔ بعد

1. `AGENTS.md` امروز را روی ریپو واقعی خود بکارید و دست‌کم سه کار با عامل انجام دهید.

   هر بار عامل خطا کرد، یک قاعده اضافه کنید و کامیت کنید تا سیر تکامل فایل دیده شود.

2. سه کار واقعی از پروژهٔ خود را روی طیف بنشانید و برای هرکدام سازوکار راستی‌آزمایی متناسب پیشنهاد دهید.

   یک صفحه، تحویل در آغاز جلسهٔ دوم.

3. مقالهٔ ترجمه‌شدهٔ جلسهٔ اول را کامل بخوانید.

   جلسهٔ دوم با فرض خواندن آن شروع می‌شود.

جلسهٔ بعد

ابزارهای عامل و تعامل‌پذیری

عامل چگونه با جهان بیرون حرف می‌زند؟
فراخوانی تابع، پروتکل زمینهٔ مدل (MCP) و گفت‌وگوی عامل با عامل.

پرسشی دارید؟ همین حالا بپرسید.